ORDER FLOW ENGINE

Complete Indicator Calculation Formulas

Every formula, algorithm, variable definition, and worked example for every indicator in the Order Flow Engine

v1.0 | May 2026 | Engineering Reference

1. Tick Classification — Lee-Ready Algorithm

Every order flow metric begins with tick classification: determining whether each trade was initiated by an aggressive buyer (ask-side) or aggressive seller (bid-side). The Lee-Ready Algorithm (Lee & Ready, Journal of Finance, 1991) is the industry standard.

1.1 Lee-Ready Algorithm — Step-by-Step

Input Variables

P_t	= Price of the current trade
M_t	= Midpoint of prevailing bid-ask spread = (Bid_t + Ask_t) / 2
Bid_t	= Best bid price at time of trade
Ask_t	= Best ask price at time of trade
P_{t-1}	= Price of the immediately preceding trade

Rule 1 — Quote Rule (applied first)

```
IF  P_t > M_t  →  CLASSIFY as  ASK  (aggressive buyer)
IF  P_t < M_t  →  CLASSIFY as  BID  (aggressive seller)
IF  P_t = M_t  →  INDETERMINATE → apply Rule 2
```

Rule 2 — Tick Rule (applied when Quote Rule is indeterminate)

```
IF  P_t > P_{t-1}  →  CLASSIFY as  ASK  (uptick = buyer initiated)
IF  P_t < P_{t-1}  →  CLASSIFY as  BID  (downtick = seller initiated)
IF  P_t = P_{t-1}  →  CLASSIFY as  LAST (zero-tick = carry forward
previous classification)
```

IQFeed Direct Aggressor Override (CME & ICE only)

```
IF  TradeAggressor_field = 1  →  CLASSIFY as  ASK  (exchange-
confirmed buyer)
IF  TradeAggressor_field = 2  →  CLASSIFY as  BID  (exchange-
confirmed seller)
IF  TradeAggressor_field = 0  →  Apply Lee-Ready above
```

★ *Exchange-direct aggressor classification (IQFeed, CME/ICE) achieves 100% accuracy. Lee-Ready achieves ~85-99% accuracy depending on market conditions. Always prefer exchange-direct when available.*

Result Mapping to Volume Buckets

```
Classification = ASK  →  ask_vol[price] += trade_volume
Classification = BID  →  bid_vol[price] += trade_volume
total_vol[price]     = ask_vol[price] + bid_vol[price]
```

2. Delta Calculations

Delta is the net difference between aggressive buying volume (ask-side) and aggressive selling volume (bid-side). It is the most fundamental order flow metric.

2.1 Bar Delta

Formula

```
BarDelta  =  SUM(ask_vol[i])  -  SUM(bid_vol[i])
          =  TotalAskVol_bar  -  TotalBidVol_bar

Where the sum is over all price levels i within the bar.
```

BarDelta = Net aggressive buying pressure for one bar. Positive = net buying, Negative = net selling.

ask_vol[i] = Ask-side (aggressive buyer) volume at price level i

bid_vol[i] = Bid-side (aggressive seller) volume at price level i

Worked Example (5-minute ES bar)

Price Level	bid_vol	ask_vol	Level Delta
4502.25	120	180	+60
4502.50	200	150	-50
4502.75	90	310	+220
4503.00	300	100	-200
TOTAL	710	740	BarDelta = 740 - 710 = +30

2.2 Max Delta & Min Delta

Formula

```
MaxDelta  =  max(RunningDelta_t)  for all ticks t within the bar
MinDelta  =  min(RunningDelta_t)  for all ticks t within the bar

RunningDelta_t  =  SUM of per-tick delta from bar_open to tick t
                =  RunningDelta_{t-1} + delta(tick_t)

delta(tick_t)  =  +volume  if tick classified as ASK
               =  -volume  if tick classified as BID
```

★ *MaxDelta and MinDelta track the peak and trough of the running delta within the bar — measuring how far the delta swung in each direction before the bar closed. A bar with MaxDelta=+500 and BarDelta=+50 tells a very different story than a bar with MaxDelta=+500 and BarDelta=+490.*

2.3 Delta Percentage (Delta/Volume %)

Formula

```
DeltaPct  =  (BarDelta / TotalBarVolume) x 100

TotalBarVolume  =  SUM(ask_vol[i]) + SUM(bid_vol[i])  for all i
                =  TotalAskVol_bar + TotalBidVol_bar
```

Example: BarDelta = +30, TotalBarVolume = 1450 → DeltaPct = (30/1450) × 100 = +2.07%

★ *DeltaPct normalises delta for comparison across bars of different sizes. A delta of +30 on a 100-contract bar is more significant than +30 on a 10,000-contract bar. DeltaPct makes these comparable.*

2.4 Cumulative Delta (CVD)

Formula

```
CVD_n  =  SUM(BarDelta_i)  for i = 1 to n
       =  CVD_{n-1}  +  BarDelta_n

CVD resets to 0 at each session open (session_open_ts).

CVD_1  =  BarDelta_1  (first bar of session)
```

CVD_n	= Cumulative Delta after bar n. Running session total.
BarDelta_n	= Delta of bar n

Example session: Bar1 delta=+150, Bar2 delta=-80, Bar3 delta=+220 → CVD after Bar3 = 150 + (-80) + 220 = +290

★ *CVD divergence: if Price makes a new high while CVD makes a lower high, buying pressure is weakening — potential reversal signal. If Price makes a new low while CVD makes a higher low, selling pressure is weakening — potential bounce signal.*

2.5 Delta Divergence Detection

Standard Divergence Formula

```
BullishDivergence  =  Price_n < Price_{n-k}  AND  CVD_n > CVD_{n-k}
BearishDivergence  =  Price_n > Price_{n-k}  AND  CVD_n < CVD_{n-k}

Where k = lookback period (default k = 1 to 5 bars)
Price_n = close price of bar n
CVD_n   = cumulative delta at bar n close
```

Bar-Level Delta Divergence (Valtos Webinar 18)

```
BarDeltaDivergence_Bullish  =  BarClose < BarOpen  AND  BarDelta > 0
  (Bearish candle + positive delta → sell pressure failed to close
price lower)

BarDeltaDivergence_Bearish  =  BarClose > BarOpen  AND  BarDelta < 0
  (Bullish candle + negative delta → buy pressure failed to close
price higher)
```

★ *Bar-level delta divergence is a micro-divergence within a single bar. Trade in the direction of the candle color — the candle won despite adverse delta, showing hidden strength.*

3. Imbalance Detection

Imbalances occur when aggressive volume on one side of the market significantly exceeds the opposing side at a price level. The diagonal comparison is the defining characteristic of footprint chart imbalance detection.

3.1 Single Buying Imbalance

Formula — Diagonal Comparison

```
BuyingImbalance at price P  =  TRUE

  IF  ask_vol[P]  >=  ImbalanceRatio  x  bid_vol[P - 1_tick]
  AND bid_vol[P - 1_tick]  >  0   (avoid division by zero)

Where:
  ask_vol[P]       = ask-side volume at current price level P
  bid_vol[P-1tick] = bid-side volume at ONE TICK BELOW P
  ImbalanceRatio   = user-configurable threshold (default 3.0 = 300%)
```

Formula — Single Selling Imbalance

```
SellingImbalance at price P  =  TRUE

  IF  bid_vol[P]  >=  ImbalanceRatio  x  ask_vol[P + 1_tick]
  AND ask_vol[P + 1_tick]  >  0   (avoid division by zero)

Where:
  bid_vol[P]       = bid-side volume at current price P
  ask_vol[P+1tick] = ask-side volume at ONE TICK ABOVE P
```

⚠️ *The comparison is DIAGONAL — not horizontal. ask_vol at price P is compared against bid_vol at price P-1_tick, NOT against bid_vol at price P. This diagonal comparison is what distinguishes footprint imbalance from simple volume dominance.*

Common Ratio Thresholds

Ratio Setting	Sensitivity	Typical Use
2.5x (250%)	High — detects more imbalances	Scalping on 30s or 1min bars; active markets
3.0x (300%)	Standard — industry default	5-minute bars; balanced signal quality
4.0x (400%)	Low — only strong imbalances	15-minute or 30-minute bars; high-conviction only

5.0x (500%)	Very selective	Macro-level analysis; daily/weekly context only

3.2 Stacked Imbalance

Formula

```
StackedBuyImbalance  =  TRUE

  IF  consecutive count of BuyingImbalance = TRUE
      at vertically adjacent price levels  >=  StackedMinCount

  i.e.  BuyingImbalance[P]   = TRUE
        BuyingImbalance[P+1] = TRUE
        BuyingImbalance[P+2] = TRUE   (minimum 3, default)
        ...

ZoneHigh  =  highest price level in the consecutive imbalance chain
ZoneLow   =  lowest  price level in the consecutive imbalance chain
ZoneStrength  =  count of consecutive imbalances  x
avg(ImbalanceRatio)
ImbalanceRatio_avg  =  MEAN of actual ratios at each level in the
stack
```

★ *Stacked imbalances persist across bars as support/resistance zones. Zone_ID is assigned on first detection. The zone expands if subsequent bars add more imbalances at adjacent levels. The zone is resolved (deactivated) when price trades through it.*

3.3 Imbalance Ratio (Actual)

Formula

```
ActualRatio_Buy[P]   =  ask_vol[P]       / bid_vol[P - 1_tick]
ActualRatio_Sell[P]  =  bid_vol[P]       / ask_vol[P + 1_tick]

ImbalanceStrengthScore  =  CLAMP((ActualRatio / ImbalanceRatio) x 5,
1, 10)

Where CLAMP(x, min, max) limits the value to [min, max] range.
```

Example: ImbalanceRatio=3.0, ActualRatio=7.2 → StrengthScore = CLAMP((7.2/3.0)×5, 1, 10) = CLAMP(12.0, 1, 10) = 10 (maximum strength)

3.4 Absorption Detection

Formula

```
Absorption at price level P  =  TRUE

  IF  total_vol[P]  >=  AbsorptionVolThreshold
  AND  |delta[P]|   <=  AbsorptionDeltaRatio  x  total_vol[P]

  i.e.  total_vol[P]  >=  500   (default threshold: 500 contracts)
        |ask_vol[P] - bid_vol[P]|  <=  0.10  x  total_vol[P]
        (net delta is less than 10% of total volume — near-zero net
direction)

AbsorptionDeltaRatio  =  user-configurable (default 0.10 = 10%)
```

total_vol[P]	= ask_vol[P] + bid_vol[P] — total volume at the level
|delta[P]|	= absolute value of ask_vol[P] - bid_vol[P]

★ *Absorption requires HIGH volume AND near-zero net delta. This distinguishes it from consolidation (low volume AND near-zero delta). The high volume proves institutional inventory transfer is occurring.*

4. Volume Profile Calculations

4.1 Profile Construction

Algorithm

```
1. Collect all ticks in the profile period [T_start, T_end]
2. For each tick:
     profile[price_bucket]  +=  tick_volume
     where price_bucket = ROUND(tick_price / tick_size) x tick_size
3. result: profile[] = histogram of volume at each price level
```

Price Bucket Width

```
tick_size  =  minimum price increment of the instrument
            = 0.25 pts for ES (= $12.50/contract)
            = 0.01 for NYSE stocks
            = 0.0001 for FX pairs

profile[bucket] accumulates ALL volume traded within that tick_size
increment.
```

4.2 Point of Control (POC)

Formula

```
POC  =  argmax(profile[price])
      = price level with the HIGHEST total_vol in the profile period

POC_volume  =  max(profile[price])  for all prices in period

If two price levels have equal volume:
  POC  =  lower of the two prices (conservative, closer to value)
```

4.3 Value Area Calculation

Algorithm (Industry Standard — Two-Price-Level Expansion)

```
Step 1:  target_vol  =  TotalProfileVolume  x  ValueAreaPct  (default
0.70)

Step 2:  accumulated_vol  =  profile[POC]
         VA_high  =  POC
         VA_low   =  POC

Step 3:  WHILE accumulated_vol < target_vol:

           upper_2  =  profile[VA_high + 1_tick] + profile[VA_high +
2_tick]
           lower_2  =  profile[VA_low  - 1_tick] + profile[VA_low  -
2_tick]

           IF upper_2 >= lower_2:
             VA_high         += 2_ticks
             accumulated_vol += upper_2
           ELSE:
             VA_low          -= 2_ticks
             accumulated_vol += lower_2

Step 4:  VAH  =  VA_high
         VAL  =  VA_low
         VA%_actual  =  accumulated_vol / TotalProfileVolume x 100
```

★ *The two-price-level expansion (comparing 2 levels above vs 2 levels below) is the industry standard used by Trading Technologies X_STUDY, Sierra Chart, and NinjaTrader. Some platforms expand one level at a time — results will differ slightly.*

Worked Example

Profile total volume = 10,000 contracts. Target = 7,000 (70%). POC volume = 2,100.

Iteration 1: upper_2=800+600=1400, lower_2=400+500=900 → expand up. VA: [POC, POC+2]. Accumulated=3500.

Iteration 2: upper_2=500+300=800, lower_2=900+700=1600 → expand down. VA: [POC-2, POC+2]. Accumulated=5100.

Continue until accumulated ≥ 7000. Final VAH and VAL define the value area.

4.4 High Volume Node (HVN) Detection

Formula

```
mean_vol   =  TotalProfileVolume / NumberOfPriceLevels
```

```
HVN[price] = TRUE IF profile[price] >= HVNMultiplier x
mean_vol

HVNMultiplier = user-configurable (default 1.5 = 150% of mean)

HVN Strength Score = profile[price] / mean_vol
```

4.5 Low Volume Node (LVN) Detection

Formula

```
LVN[price] = TRUE IF profile[price] <= LVNMultiplier x
mean_vol

LVNMultiplier = user-configurable (default 0.5 = 50% of mean)

LVN Gap = consecutive price levels where LVN[price] = TRUE
LVN Width = number of consecutive LVN levels
```

4.6 Volume Profile Shape Classification

Formula

```
Define:
  profile_mean   = SUM(price[i] x profile[i]) / TotalProfileVolume
  profile_stdev  = SQRT( SUM((price[i] - mean)^2 x profile[i]) /
TotalVol )
  poc_percentile = (POC - ProfileLow) / (ProfileHigh - ProfileLow)
  tail_pct_lower = volume below (POC - 1.5 x profile_stdev) /
TotalVol
  tail_pct_upper = volume above (POC + 1.5 x profile_stdev) /
TotalVol

D-Profile   = poc_percentile in [0.35, 0.65] AND profile_stdev <
threshold
P-Profile   = poc_percentile > 0.60 AND tail_pct_lower > 0.15
b-Profile   = poc_percentile < 0.40 AND tail_pct_upper > 0.15
Thin-Profile = profile_stdev > 2.0 x mean_stdev_for_this_instrument
```

5. VWAP — Volume Weighted Average Price

5.1 Core VWAP Formula

Formula

```
VWAP_n  =  SUM(Price_i x Volume_i)  /  SUM(Volume_i)
        = CumulativePriceVolume_n  /  CumulativeVolume_n

Where i = 1 to n  (all trades from anchor point to current tick n)

Incremental update (efficient — no full recalculation each tick):

  CumPV_n  =  CumPV_{n-1}  +  (Price_n  x  Volume_n)
  CumVol_n =  CumVol_{n-1} +  Volume_n
  VWAP_n   =  CumPV_n  /  CumVol_n
```

Price_i = Typical Price of trade i = Last traded price (tick-level VWAP)

Volume_i = Number of contracts/shares at trade i

★ *For bar-level VWAP (when using bar data rather than tick data): Typical Price = (High + Low + Close) / 3. The OFE engine uses tick-level prices for maximum precision since tick data is available.*

5.2 VWAP Anchor Types

Anchor Type	Anchor Point (T_start)	Reset Behavior
Daily VWAP	Session open time (e.g. 09:30:00 ET for ES)	Resets each session open
Weekly VWAP	Monday session open time	Resets each Monday
Monthly VWAP	First trading day of month, session open	Resets each month
Yearly VWAP	First trading day of year, session open	Resets each January
Anchored VWAP	User-selected timestamp or price event	Does not auto-reset — persists until removed
Session VWAP	Same as Daily VWAP for equities; Globex open for futures	Configurable per instrument

```
Anchored VWAP formula is identical to core VWAP:
  VWAP_anchored(T_anchor, t)  =  SUM(P_i x V_i) / SUM(V_i)
  for all i where T_anchor <= timestamp_i <= t
```

5.3 VWAP Standard Deviation Bands

Formula

```
Step 1: Calculate variance at each tick n:
  Variance_n  =  CumPV2_n / CumVol_n  -  VWAP_n ^ 2

  CumPV2_n  =  CumPV2_{n-1}  +  (Price_n ^ 2  x  Volume_n)
  (tracks cumulative sum of price-squared weighted by volume)

Step 2: Standard Deviation:
  StdDev_n  =  SQRT(Variance_n)

Step 3: Bands:
  Band_+k  =  VWAP_n  +  (k  x  StdDev_n)    upper band, k-th
deviation
  Band_-k  =  VWAP_n  -  (k  x  StdDev_n)    lower band, k-th
deviation

Standard values: k = 1, 2, 3
```

★ *This is a VOLUME-WEIGHTED standard deviation — not the same as simple standard deviation. It weights the variance by the volume traded at each price, making it more sensitive to high-volume price levels. This is what Dale's book refers to as the 'VWAP deviation band'.*

Example: VWAP=4502.50, StdDev=8.25 → Band+1=4510.75, Band-1=4494.25, Band+2=4519.00, Band-2=4486.00

5.4 VWAP Signal Conditions

VWAP Reaction Signal

```
VWAPTouch  =  ABS(CurrentPrice - VWAP_n)  <=
VWAPTouchThreshold_ticks x tick_size

VWAP_Reaction_Long   =  VWAPTouch = TRUE
                        AND  CurrentPrice_prev > VWAP_n  (approaching
from above)
                        AND  BarDelta > 0  (positive delta
confirmation)
```

```
VWAP_Reaction_Short  =  VWAPTouch = TRUE
                        AND  CurrentPrice_prev < VWAP_n  (approaching
from below)
                        AND  BarDelta < 0  (negative delta
confirmation)
```

VWAP Rotation Signal

```
VWAP_Rotation_Long   =  CurrentPrice  <=  Band_-1
                        AND  BarDelta  >  0          (buying at
lower band)
                        AND  CVD divergence confirmed (optional)

VWAP_Rotation_Short  =  CurrentPrice  >=  Band_+1
                        AND  BarDelta  <  0          (selling at
upper band)
```

6. Commitment of Traders (COT) — Intrabar

Note: This is Valtos's proprietary intrabar COT concept — NOT the CFTC weekly COT report. It identifies the price level within a bar where institutional commitment was highest.

6.1 COT Price Formula

Formula

```
COT_price  =  argmax(total_vol[price])  within the current bar
           =  price level with maximum  ask_vol[P] + bid_vol[P]

COT_volume =  max(total_vol[price])  within the current bar
           =  max(ask_vol[P] + bid_vol[P])  for all P in bar

COT_delta  =  ask_vol[COT_price] - bid_vol[COT_price]
```

COT Position Interpretation

```
COT_position_ratio  =  (COT_price - BarLow) / (BarHigh - BarLow)

IF  COT_position_ratio  <  0.33  (COT near bar low):
    AND  BarClose > BarOpen  (bullish bar):
    → Strong passive buyers at lows — bullish signal

IF  COT_position_ratio  >  0.67  (COT near bar high):
    AND  BarClose < BarOpen  (bearish bar):
    → Strong passive sellers at highs — bearish signal

IF  COT_position_ratio  in [0.33, 0.67]  (COT in middle):
    → Neutral — two-sided market facilitation
```

7. Orderflows Pulse — Composite Score Formula

The Pulse is a 7-variable composite indicator. Each variable produces a sub-score from 0-100. The composite score is a weighted sum. A score >= 70 is considered high-conviction.

7.1 Variable Sub-Scores

Variable 1: Order Flow Score

```
OF_score  =  CLAMP(ABS(BarDelta) / BarVolume x 200, 0, 100)

Interpretation: measures how aggressively one side is dominating.
0 = perfectly balanced. 100 = 50%+ delta/volume ratio (very
aggressive).
```

Variable 2: Delta Score

```
Delta_score  =  CLAMP(ABS(BarDelta) / MaxDelta_session x 100, 0, 100)

Interpretation: how large is this bar's delta relative to today's
maximum?
100 = this is the most aggressive delta bar of the session.
```

Variable 3: POC Score

```
POC_position  =  ABS(BarClose - COT_price) / (BarHigh - BarLow)
POC_score     =  CLAMP((1 - POC_position) x 100, 0, 100)

Interpretation: how close is bar close to the bar's COT price?
100 = bar closed exactly at the COT level (maximum institutional
alignment).
0 = bar closed far from the COT level.
```

Variable 4: Imbalance Score

```
ImbalanceCount  =  number of imbalance cells in this bar
Imbalance_score =  CLAMP(ImbalanceCount / 5 x 100, 0, 100)

Interpretation: more imbalances = stronger imbalance score.
Stacked imbalances get bonus: score x 1.5 if StackedCount >=
StackedMinCount.
```

Variable 5: Volume Score

```
AvgBarVolume_session  =  CumVolume_session / BarCount_session
Volume_score  =  CLAMP(BarVolume / AvgBarVolume_session x 100, 0, 100)

Interpretation: is this bar's volume above average?
100 = bar volume is >= average (sufficient participation for valid signal).
Below average volume → score < 100 → reduces composite score.
```

Variable 6: Price Action Score

```
BarRange     =  BarHigh - BarLow
AvgRange     =  EMA(BarRange, 14)    (14-bar EMA of bar range)
CandleBody   =  ABS(BarClose - BarOpen)
BodyRatio    =  CandleBody / BarRange

PA_score  =  CLAMP(BodyRatio x (BarVolume / AvgBarVolume) x 100, 0, 100)

Interpretation: strong close relative to range + above-average volume = high PA score.
```

Variable 7: Swing Score

```
SwingHigh_N  =  highest bar high in last N bars  (default N = 10)
SwingLow_N   =  lowest  bar low  in last N bars

SwingPct  =  (BarClose - SwingLow_N) / (SwingHigh_N - SwingLow_N)

IF  PulseDirection = LONG:
  Swing_score  =  CLAMP(SwingPct x 100, 0, 100)
  (100 = bar closed at top of recent swing range — momentum up)

IF  PulseDirection = SHORT:
  Swing_score  =  CLAMP((1 - SwingPct) x 100, 0, 100)
  (100 = bar closed at bottom of recent swing range — momentum down)
```

7.2 Composite Pulse Score

Formula

```
Weights (default, sum to 1.0):
  w_OF   = 0.20   (order flow dominance)
  w_D    = 0.20   (delta strength)
  w_POC  = 0.15   (POC alignment)
  w_IMB  = 0.20   (imbalance count)
  w_VOL  = 0.10   (volume filter)
  w_PA   = 0.10   (price action)
  w_SW   = 0.05   (swing context)

PulseScore  =  w_OF  x OF_score
             + w_D   x Delta_score
             + w_POC x POC_score
             + w_IMB x Imbalance_score
             + w_VOL x Volume_score
             + w_PA  x PA_score
             + w_SW  x Swing_score

PulseScore range: 0 – 100
Signal threshold: PulseScore >= 70  →  emit Pulse signal
```

Direction Determination

```
IF  BarDelta > 0  AND  BarClose > BarOpen:
    PulseDirection  =  LONG

IF  BarDelta < 0  AND  BarClose < BarOpen:
    PulseDirection  =  SHORT

IF  signs conflict (BarDelta > 0 but BarClose < BarOpen):
    PulseDirection  =  NEUTRAL  (Bar Delta Divergence — no Pulse
signal)

Pulse signal only fires if PulseScore >= threshold AND Direction !=
NEUTRAL.
```

8. Orderflows Turns — Turning Point Score

The Turns indicator uses 6 variables to detect market turning points. Scores from 0-6 (one point per variable alignment). A score >= 3 is a valid Turn signal; >= 5 is high-conviction.

8.1 Six-Variable Turning Point Score

```
TurnScore  =  V1_poc + V2_bid_ask + V3_volume + V4_price_action +
V5_delta + V6_swing

Each variable contributes 1 point if its condition is met:

V1_poc (POC):
  +1  IF  ABS(BarHigh - COT_price) <= 2_ticks  (turn at high,
bearish)
      OR   ABS(BarLow - COT_price) <= 2_ticks   (turn at low,
bullish)

V2_bid_ask (Volume at extremes):
  +1  IF  ask_vol[BarHigh_price] < VolumeExhaustThreshold
      OR   bid_vol[BarLow_price]  < VolumeExhaustThreshold
  (very few contracts at bar extreme = last buyer/seller =
exhaustion)

V3_volume (Sufficient total volume):
  +1  IF  BarVolume  >=  MinTurnVolume  (default: 0.5 x AvgBarVolume)

V4_price_action (Reversal candle):
  Bullish Turn: +1  IF  BarClose > BarOpen + (BarRange x 0.5)
  Bearish Turn: +1  IF  BarClose < BarOpen - (BarRange x 0.5)
  (bar closed in the reversal direction by >50% of range)

V5_delta (Delta confirmation):
  Bullish Turn: +1  IF  BarDelta > 0  (positive delta supports
bounce)
  Bearish Turn: +1  IF  BarDelta < 0  (negative delta supports drop)

V6_swing (At swing high/low):
  +1  IF  BarHigh >= SwingHigh_N - 2_ticks  (bearish turn at swing
high)
      OR   BarLow  <= SwingLow_N  + 2_ticks  (bullish turn at swing
low)
```

Turn Signal Emission

```
IF  TurnScore >= 3  AND  TurnScore <= 4:  TurnStrength = MODERATE
IF  TurnScore >= 5:                        TurnStrength = STRONG

TurnDirection:
  BULLISH_TURN  =  BarLow is at extreme AND V4 = bullish reversal
candle
  BEARISH_TURN  =  BarHigh is at extreme AND V4 = bearish reversal
candle
```

9. Orderflows Ratio — Top Heavy / Bottom Heavy

9.1 Formula

```
For detecting BOTTOM HEAVY (Big Buyer at Low):

  LowPriceBidVol  =  bid_vol[BarLow_price]
  LowPriceAskVol  =  ask_vol[BarLow_price]
  BarLow_total    =  LowPriceBidVol + LowPriceAskVol

  BottomHeavyRatio  =  LowPriceBidVol / max(LowPriceAskVol, 1)

  BottomHeavy  =  TRUE
    IF  BottomHeavyRatio  >=  RatioThreshold  (default 3.0 = 300%)
    AND BarLow_total      >=  MinRatioVolume  (default 100 contracts)
    AND BarLow  is within  N_ticks of SessionLow  (default N = 5)

For detecting TOP HEAVY (Big Seller at High):

  TopHeavyRatio  =  HighPriceAskVol / max(HighPriceBidVol, 1)

  TopHeavy  =  TRUE
    IF  TopHeavyRatio     >=  RatioThreshold
    AND BarHigh_total     >=  MinRatioVolume
    AND BarHigh  is within  N_ticks of SessionHigh
```

★ *One signal fires per bar maximum. BottomHeavy and TopHeavy are mutually exclusive per bar — a bar cannot be both simultaneously. If both conditions are met (unusual), the stronger ratio takes priority.*

10. Single Prints — Last Buyer / Last Seller

10.1 Formula

```
LastBuyer_at_High = TRUE
  IF total_vol[BarHigh_price] <= SinglePrintThreshold (default 2
contracts)
  AND ask_vol[BarHigh_price]   >  0   (someone bought at the high)
  AND BarHigh   >= PriorBarHigh  (new high or equal high)

LastSeller_at_Low = TRUE
  IF total_vol[BarLow_price]  <= SinglePrintThreshold
  AND bid_vol[BarLow_price]    >  0   (someone sold at the low)
  AND BarLow    <= PriorBarLow  (new low or equal low)

ExhaustionScore = 1.0 - (total_vol[extreme_price] /
AvgVol_at_extremes)
  where AvgVol_at_extremes = EMA(total_vol[BarHigh_price], 10)

UnfinishedAuction = TRUE
  IF ask_vol[BarHigh_price] > 0  (ask still showing at high →
unfinished)
  OR  bid_vol[BarLow_price]  > 0  (bid still showing at low  →
unfinished)
```

11. Delta Surge

11.1 Formula

```
AvgDelta_N  =  EMA(ABS(BarDelta), N)   (N-bar EMA of absolute delta,
default N=14)

SurgeMagnitude  =  ABS(BarDelta) / max(AvgDelta_N, 1)

BullishSurge  =  SurgeMagnitude >= SurgeThreshold  AND  BarDelta > 0
BearishSurge  =  SurgeMagnitude >= SurgeThreshold  AND  BarDelta < 0

SurgeThreshold  =  user-configurable (default 3.0 = 300% of average
delta)

BarsBuilding  =  count of consecutive bars with ABS(BarDelta) >=
AvgDelta_N
  (how many bars in a row have above-average delta in the same
direction)
```

Example: AvgDelta_14=200, BarDelta=+750 → SurgeMagnitude = 750/200 = 3.75 → BullishSurge triggered (3.75 >= 3.0)

12. Market Sweep Detector

12.1 Formula

```
A market sweep occurs when a large aggressive order clears through multiple
consecutive price levels in rapid succession within a single bar.

SweepVolume  =  SUM(dominant_vol[P])  for P in sweep range
  where dominant_vol[P] = ask_vol[P] for bullish sweep, bid_vol[P] for bearish

SweepPriceLevels  =  count of consecutive price levels cleared in one direction
SweepSpeed  =  SweepPriceLevels / SweepTimespan_ms  (levels per millisecond)

Sweep_detected  =  TRUE
  IF  SweepVolume      >=  SweepVolumeThreshold   (default 500 contracts)
  AND SweepPriceLevels >=  SweepLevelThreshold    (default 5 price levels)
  AND SweepSpeed       >=  SweepSpeedThreshold    (default 2 levels/ms)
  AND all levels cleared in same direction (no opposing volume > 10% of sweep)
```

13. Trapped Traders Detection

13.1 Formula

```
TrappedSellers  =  TRUE
  IF  SellingImbalance detected at price P
  AND P is within 2 ticks of BarLow
  AND BarClose > BarOpen + (BarRange x 0.5)   (bar closed in upper
half)
  (Sellers who sold the low are now trapped — price closed strongly
above them)

TrappedBuyers  =  TRUE
  IF  BuyingImbalance detected at price P
  AND P is within 2 ticks of BarHigh
  AND BarClose < BarOpen - (BarRange x 0.5)   (bar closed in lower
half)
  (Buyers who bought the high are now trapped — price closed strongly
below them)

TrapStrength  =  ImbalanceRatio_at_level x (BarRange / AvgBarRange)
```

14. POC Slingshot

14.1 Formula

```
POC_Slingshot  =  TRUE  (BULLISH)
  IF  PriorBar:  BarClose within N_ticks of SessionPOC  (price near
POC)
  AND CurrentBar:
      BarClose  > SessionPOC  + SlingDistanceTicks x tick_size
      (price broke away from POC by at least SlingDistance ticks)
  AND BarDelta  > SlingDeltaThreshold x AvgBarDelta
      (strong delta supporting the break — default 1.5x average)
  AND BarVolume > SlingVolumeThreshold x AvgBarVolume
      (above-average volume confirming the break — default 1.2x
average)

SlingDistanceTicks  =  user-configurable (default 4 ticks for ES)

POC_Slingshot_Bearish  =  symmetric definition — break BELOW POC
```

15. Complete Formula Reference Summary

Indicator	Core Formula	Key Variables	Default Threshold
Bar Delta	ask_vol_sum - bid_vol_sum per bar	ask_vol[P], bid_vol[P] at each price level	N/A — raw value
Delta Percentage	BarDelta / TotalBarVolume × 100	BarDelta, TotalBarVolume	N/A — raw %
Max/Min Delta	max/min of RunningDelta_t within bar	RunningDelta_t per tick	N/A — tracking extremes
Cumulative Delta	CVD_n = CVD_{n-1} + BarDelta_n	BarDelta sequence	Resets at session open
VWAP	CumPV / CumVol from anchor	CumPV = Σ(P×V), CumVol = ΣV	Session open as anchor
VWAP Std Dev	SQRT(CumPV² / CumVol - VWAP²)	CumPV², CumVol, VWAP	k=1,2 for bands
Lee-Ready	Quote Rule → Tick Rule	P_t, M_t, Bid_t, Ask_t, P_{t-1}	P_t vs M_t first
Buying Imbalance	ask_vol[P] ≥ ratio × bid_vol[P-1tick]	ask_vol[P], bid_vol[P-1]	ratio = 3.0 (300%)
Selling Imbalance	bid_vol[P] ≥ ratio × ask_vol[P+1tick]	bid_vol[P], ask_vol[P+1]	ratio = 3.0 (300%)
Stacked Imbalance	≥ N consecutive imbalances same direction	ImbalanceChain, N	N ≥ 3 consecutive
Absorption	total_vol ≥ thresh AND |delta| ≤ ratio × total_vol	total_vol[P], delta[P]	500 contracts, 10%
POC	argmax(profile[price])	volume histogram	Highest volume level
Value Area	Expand ±2 levels from POC until ≥ 70% volume	profile[], target_vol	70% (configurable)
HVN	profile[P] ≥ mult × mean_vol	profile[P], mean_vol	mult = 1.5
LVN	profile[P] ≤ mult × mean_vol	profile[P], mean_vol	mult = 0.5
COT Price	argmax(total_vol[P]) within bar	ask_vol[P]+bid_vol[P]	Per-bar calculation
Pulse Score	Weighted sum of 7 sub-scores (0-100)	7 variables, weights sum to 1.0	Score ≥ 70 = signal
Turns Score	Count of 6 binary variable alignments (0-6)	6 binary conditions	Score ≥ 3 = signal
Orderflows Ratio	ask_vol[High] / bid_vol[High] for TopHeavy	extreme price volumes	ratio ≥ 3.0

Single Print	total_vol[extreme] ≤ threshold	volume at bar extreme	≤ 2 contracts
Delta Surge	ABS(BarDelta) / AvgDelta_N ≥ threshold	BarDelta, EMA(|BarDelta|,14)	≥ 3.0 (300%)
Market Sweep	Sweep volume + levels + speed thresholds	vol, level count, speed	500 contracts, 5 levels
Trapped Traders	Imbalance at extreme + opposite close direction	Imbalance price, BarClose	Within 2 ticks of extreme
POC Slingshot	Break from POC + delta + volume confirmation	POC, BarDelta, BarVolume	4 ticks, 1.5× avg delta
Delta Divergence	Price new high/low + CVD opposite direction	Price_n, CVD_n vs prior bars	Configurable lookback k

END — Complete Indicator Calculation Formulas v1.0

26 indicators · All formulas with variables defined · Worked examples · Implementation notes · Complete reference table